

Q1

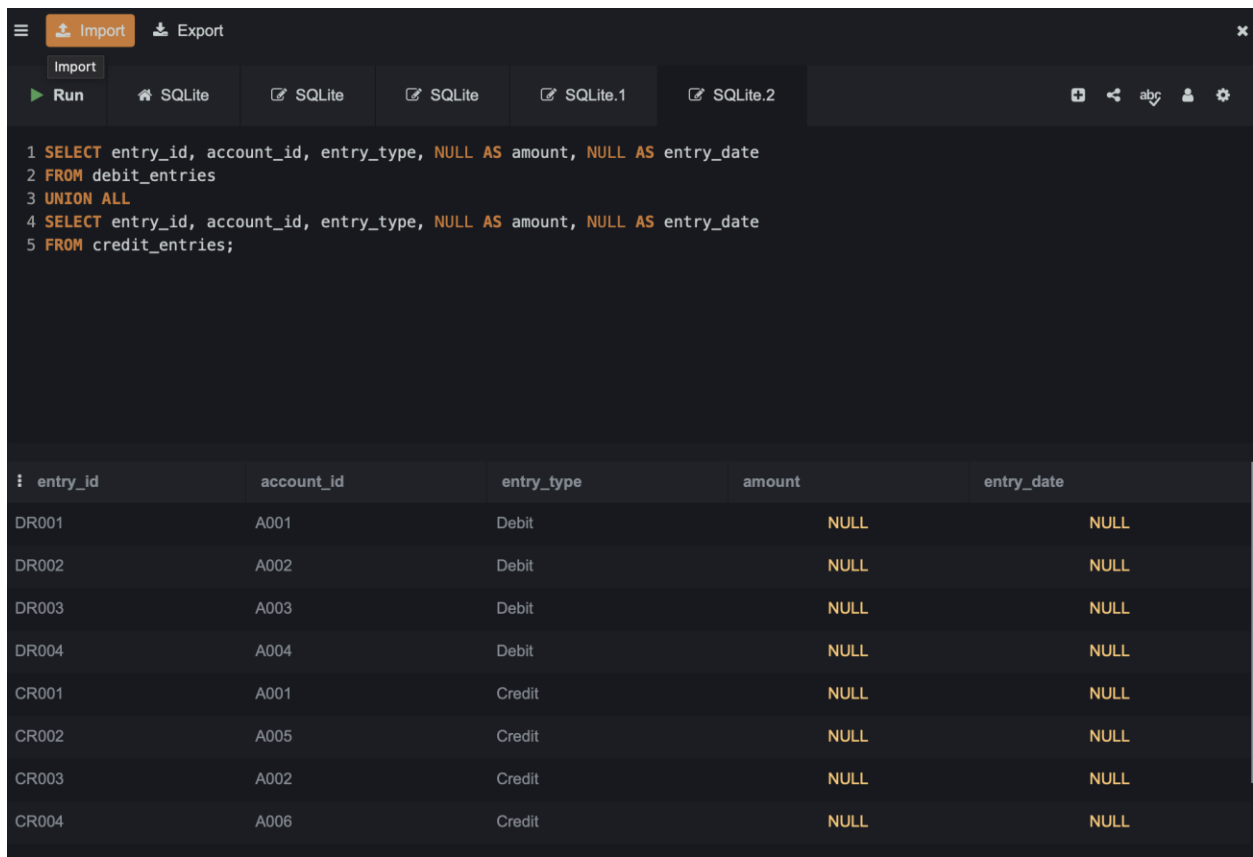
SELECT account_id, account_holder, 'Sandton' AS city

FROM branch_sandton_accounts

UNION

SELECT account_id, account_holder, 'Rosebank' AS city

FROM branch_rosebank_accounts;



The screenshot shows a SQL IDE interface with a dark theme. At the top, there are buttons for 'Import' and 'Export'. Below these are tabs for 'Run', 'SQLite', and several 'SQLite' instances. The main area contains a SQL query with line numbers 1 through 5. The query is a UNION ALL of two SELECT statements. The first SELECT statement selects from 'debit_entries' and the second from 'credit_entries'. Both SELECT statements select 'entry_id', 'account_id', 'entry_type', 'amount', and 'entry_date', with 'amount' and 'entry_date' being NULL. Below the query, the results are displayed in a table with 5 columns: 'entry_id', 'account_id', 'entry_type', 'amount', and 'entry_date'. The results show 8 rows of data, with the first 4 rows being debit entries and the next 4 rows being credit entries. All 'amount' and 'entry_date' values are NULL.

```
1 SELECT entry_id, account_id, entry_type, NULL AS amount, NULL AS entry_date
2 FROM debit_entries
3 UNION ALL
4 SELECT entry_id, account_id, entry_type, NULL AS amount, NULL AS entry_date
5 FROM credit_entries;
```

entry_id	account_id	entry_type	amount	entry_date
DR001	A001	Debit	NULL	NULL
DR002	A002	Debit	NULL	NULL
DR003	A003	Debit	NULL	NULL
DR004	A004	Debit	NULL	NULL
CR001	A001	Credit	NULL	NULL
CR002	A005	Credit	NULL	NULL
CR003	A002	Credit	NULL	NULL
CR004	A006	Credit	NULL	NULL

Q2

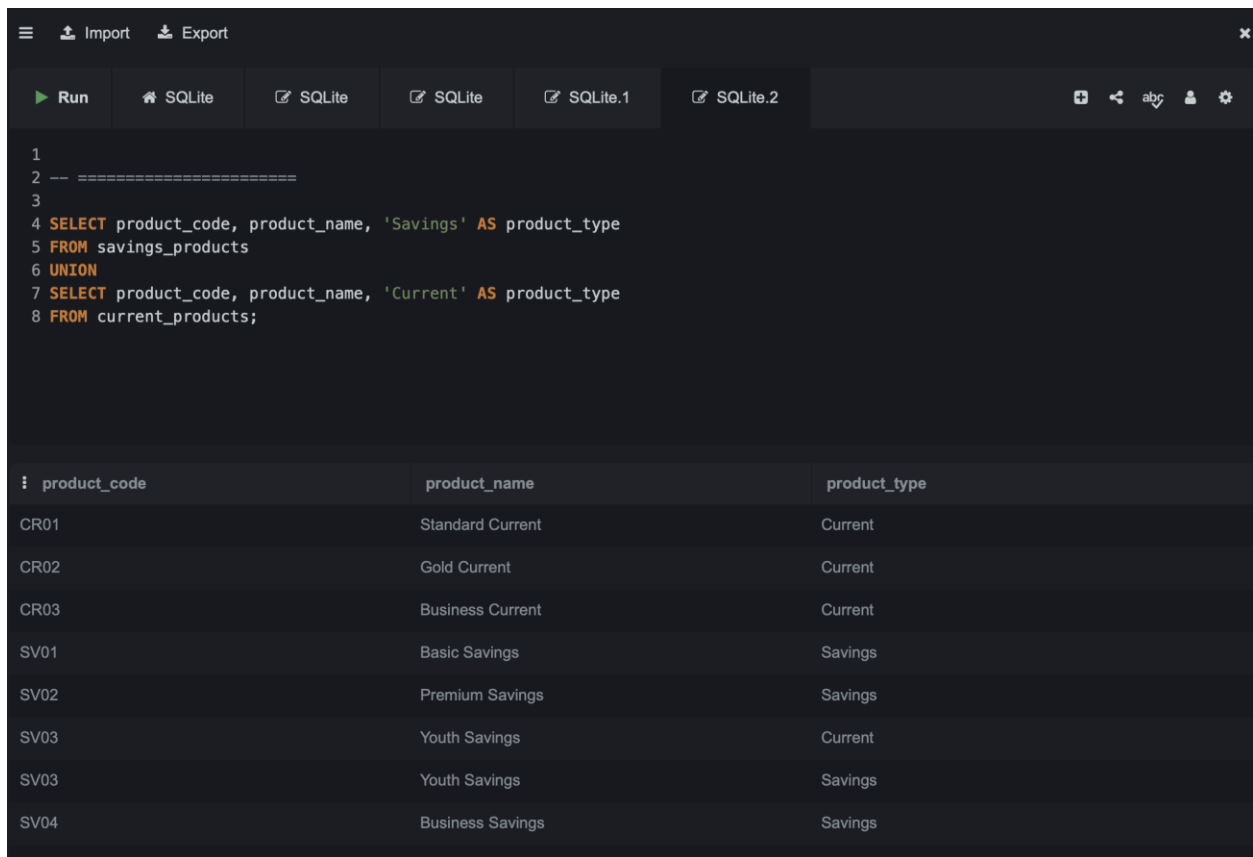
SELECT product_code, product_name, 'Savings' AS product_type

FROM savings_products

UNION

SELECT product_code, product_name, 'Current' AS product_type

FROM current_products;



The screenshot shows a SQL IDE interface with a dark theme. At the top, there are buttons for 'Run', 'SQLite', and 'Export'. Below the buttons, a SQL query is entered in a text area. The query is a UNION of two SELECT statements. The first SELECT statement selects from 'savings_products' and the second SELECT statement selects from 'current_products'. The results of the query are displayed in a table below the text area. The table has three columns: 'product_code', 'product_name', and 'product_type'. The results show 10 rows of data, including 'Standard Current', 'Gold Current', 'Business Current', 'Basic Savings', 'Premium Savings', 'Youth Savings', and 'Business Savings'.

```
1
2 -- =====
3
4 SELECT product_code, product_name, 'Savings' AS product_type
5 FROM savings_products
6 UNION
7 SELECT product_code, product_name, 'Current' AS product_type
8 FROM current_products;
```

product_code	product_name	product_type
CR01	Standard Current	Current
CR02	Gold Current	Current
CR03	Business Current	Current
SV01	Basic Savings	Savings
SV02	Premium Savings	Savings
SV03	Youth Savings	Current
SV03	Youth Savings	Savings
SV04	Business Savings	Savings

Q3

SELECT staff_id, staff_name, NULL AS email

FROM retail_banking_staff

UNION

SELECT staff_id, staff_name, NULL AS email

FROM corporate_banking_staff;

Import Export Run SQLite SQLite SQLite SQLite.1 SQLite.2 abc		
<pre> 1 SELECT staff_id, staff_name, NULL AS email 2 FROM retail_banking_staff 3 UNION 4 SELECT staff_id, staff_name, NULL AS email 5 FROM corporate_banking_staff; </pre>		
staff_id	staff_name	email
S001	Mpho Radebe	NULL
S002	Brian Tshabalala	NULL
S003	Aisha Patel	NULL
S004	Kabelo Moabelo	NULL
S005	Nandi Dube	NULL
S006	Sipho Khumalo	NULL

Q4

```
SELECT city_code, city_name, NULL AS region
FROM mobile_branch_cities

UNION

SELECT city_code, city_name, NULL AS region
FROM digital_branch_cities;
```

The screenshot shows a SQL editor with a dark theme. At the top, there are menu items: 'Import' and 'Export'. Below the menu is a toolbar with a 'Run' button and several 'SQLite' database connection tabs. The main area contains a SQL query using a UNION to combine data from two tables. Below the query, the results are displayed in a table with three columns: 'city_code', 'city_name', and 'region'. The results show six rows of data, all with 'NULL' in the 'region' column.

```

1 SELECT city_code, city_name, NULL AS region
2 FROM mobile_branch_cities
3 UNION
4 SELECT city_code, city_name, NULL AS region
5 FROM digital_branch_cities;

```

city_code	city_name	region
C01	Johannesburg	NULL
C02	Pretoria	NULL
C03	Cape Town	NULL
C04	Durban	NULL
C05	Polokwane	NULL
C06	Port Elizabeth	NULL

Q5

```

SELECT customer_id, customer_name, NULL AS segment
FROM push_notification_targets
UNION
SELECT customer_id, customer_name, NULL AS segment
FROM inapp_banner_targets;

```

The screenshot shows a SQL IDE interface with a dark theme. At the top, there are buttons for 'Import' and 'Export'. Below that is a toolbar with a 'Run' button and several 'SQLite' connection buttons. The main area contains a SQL query using a UNION to combine data from two tables. Below the query, the results are displayed in a table with three columns: customer_id, customer_name, and segment. The results show six rows of data, all with NULL in the segment column.

```

1 SELECT customer_id, customer_name, NULL AS segment
2 FROM push_notification_targets
3 UNION
4 SELECT customer_id, customer_name, NULL AS segment
5 FROM inapp_banner_targets;

```

customer_id	customer_name	segment
C1001	Nomsa Zwane	NULL
C1002	Andile Buthelezi	NULL
C1003	Fatima Mahomed	NULL
C1004	Ryno van Zyl	NULL
C1005	Thandeka Cele	NULL
C1006	Samuel Nkosi	NULL

Q6

```

SELECT txn_id AS transaction_id, account_id, amount, NULL AS transaction_date
FROM atm01_transactions

UNION ALL

SELECT txn_id AS transaction_id, account_id, amount, NULL AS transaction_date
FROM atm02_transactions;

```

The screenshot shows the SQLiteonline.com web interface. The main area displays a SQL query that unions two tables, `atm01_transactions` and `atm02_transactions`. The results are shown in a table with columns `transaction_id`, `account_id`, `amount`, and `transaction_date`. The results table contains 7 rows of data.

transaction_id	account_id	amount	transaction_date
T1001	A001	500	NULL
T1002	A002	1200	NULL
T1003	A001	300	NULL
T1004	A003	750	NULL
T1003	A001	300	NULL
T1005	A004	900	NULL
T1006	A002	450	NULL
T1007	A005	150	NULL

On the right side, there are three panels showing other SQL queries and their results:

- SQLite.2**: `SELECT customer_id, customer_name, N FROM push_notification_targets UNION SELECT custo`
- SQLite.2**: `SELECT city_code, city_name, NULL AS FROM mobile_branch_cities UNION SELECT city_code, city_n`
- SQLite.2**: `SELECT staff_id, staff_name, NULL AS FROM retail_banking_staff UNION SELECT staff_id, staff_na`
- SQLite.2**: `SELECT product_code, product_name, ' FROM savin`

Q7

```
SELECT app_id AS application_id, customer_id, loan_type, NULL AS amount_requested
FROM gauteng_loan_applications

UNION ALL

SELECT app_id AS application_id, customer_id, loan_type, NULL AS amount_requested
FROM western_cape_loan_applications;
```


⋮

ImportExport

✕

Import

Run

SQLite

SQLite

SQLite

SQLite.1

SQLite.2

+

↶

abc

👤

⚙️

```
1 SELECT complaint_id, customer_id, NULL AS category, NULL AS logged_date
2 FROM email_complaints
3 UNION ALL
4 SELECT complaint_id, customer_id, NULL AS category, NULL AS logged_date
5 FROM app_complaints;
```

complaint_id	customer_id	category	logged_date
EC001	C2001	NULL	NULL
EC002	C2002	NULL	NULL
EC003	C2003	NULL	NULL
EC004	C2004	NULL	NULL
AC001	C2005	NULL	NULL
AC002	C2001	NULL	NULL
AC003	C2006	NULL	NULL
AC004	C2007	NULL	NULL

Q9

```
SELECT payment_id, account_id, amount, NULL AS payment_date
FROM april_payments
UNION ALL
SELECT payment_id, account_id, amount, NULL AS payment_date
FROM may_payments;
```


Import Export Run SQLite SQLite SQLite SQLite.1 SQLite.2 abc				
<pre> 1 SELECT payment_id, account_id, amount, NULL AS payment_date 2 FROM april_payments 3 UNION ALL 4 SELECT payment_id, account_id, amount, NULL AS payment_date 5 FROM may_payments; </pre>				
payment_id	account_id	amount	payment_date	
PAY001	A001	12500	NULL	
PAY002	A002	4800	NULL	
PAY003	A003	9200	NULL	
PAY004	A004	3300	NULL	
PAY005	A001	12500	NULL	
PAY006	A005	7600	NULL	
PAY007	A002	5100	NULL	
PAY008	A006	2800	NULL	

Q10

```

SELECT entry_id, account_id, entry_type, NULL AS amount, NULL AS entry_date
FROM debit_entries
UNION ALL
SELECT entry_id, account_id, entry_type, NULL AS amount, NULL AS entry_date
FROM credit_entries;

```

The screenshot shows a SQL IDE interface with a dark theme. At the top, there are buttons for 'Import' and 'Export'. Below them is a toolbar with a 'Run' button and several 'SQLite' connection buttons. The main area contains a SQL query:

```

1 SELECT entry_id, account_id, entry_type, NULL AS amount, NULL AS entry_date
2 FROM debit_entries
3 UNION ALL
4 SELECT entry_id, account_id, entry_type, NULL AS amount, NULL AS entry_date
5 FROM credit_entries;

```

Below the query, the results are displayed in a table with 5 columns: entry_id, account_id, entry_type, amount, and entry_date. The results show 8 rows of data, with the first 4 rows being debit entries and the next 4 rows being credit entries. All 'amount' and 'entry_date' values are NULL.

entry_id	account_id	entry_type	amount	entry_date
DR001	A001	Debit	NULL	NULL
DR002	A002	Debit	NULL	NULL
DR003	A003	Debit	NULL	NULL
DR004	A004	Debit	NULL	NULL
CR001	A001	Credit	NULL	NULL
CR002	A005	Credit	NULL	NULL
CR003	A002	Credit	NULL	NULL
CR004	A006	Credit	NULL	NULL

Bonus Questions

Bonus 01: Use **UNION**. The goal is a *unique* list of high-risk customers — if someone appeared on both the January and February watch lists, they should be flagged once, not twice. UNION automatically removes those duplicate customer records.

Bonus 02: Use **UNION ALL**. The audit team needs to count every individual transaction event, including cases where the same transaction was recorded in two systems. Using UNION would silently drop those duplicates and undercount the true transaction volume — which would make the audit incorrect.

Bonus 03: The query fails because **the two SELECT statements return a different number of columns** — the first selects 2 columns (account_id, account_holder) while the second selects 3 (account_id, account_holder, city). UNION requires both sides to have the same column count. The fix is to add city to the first SELECT:

```
SELECT account_id, account_holder, 'Sandton' AS city
```

```
FROM branch_sandton_accounts
```

UNION

SELECT account_id, account_holder, city

FROM branch_rosebank_accounts;